Relational databases have their own programming language, called *Structured Query Language* (SQL). SQL includes structural directives for managing tables and query directives for handling the data inside the tables. One way of dealing with storage for your website is to write SQL embedded inside your Python, executing it through a connection to your database. In this case, when you query the table with song data, you get back a series of rows from the song table. Any reference to a band identifier would mean another SQL query, this time to the band table.

Creating tables means writing scripts in SQL. If you decide to add a *composer* column to the song table, that means another SQL script. It doesn't take long before you are writing a lot of SQL, but fortunately, there is an alternative.

ORM is a programming technique that maps content in a relational database to object-oriented code. Instead of thinking in terms of tables, you write classes representing the data in your project. A `Musician` class contains attributes for first name, last name, and date of birth, and then the ORM maps this class to tables in the database. This abstraction means you don't need to write a single line of SQL.

The downside of this mapping is you are now defining the structure of your data in two places: your classes and the database. Adding a stage name to your musician means changing the class but also keeping the database in sync. Django's ORM includes a table management feature called *migration*. The migration process tracks changes you make to your classes and writes the corresponding SQL scripts for you.

Figure 4.2 shows how the ORM fits into your Django code. You build models in your app; those models are managed through migration scripts; and then inside of your views, you interact with the database to populate content for your pages.

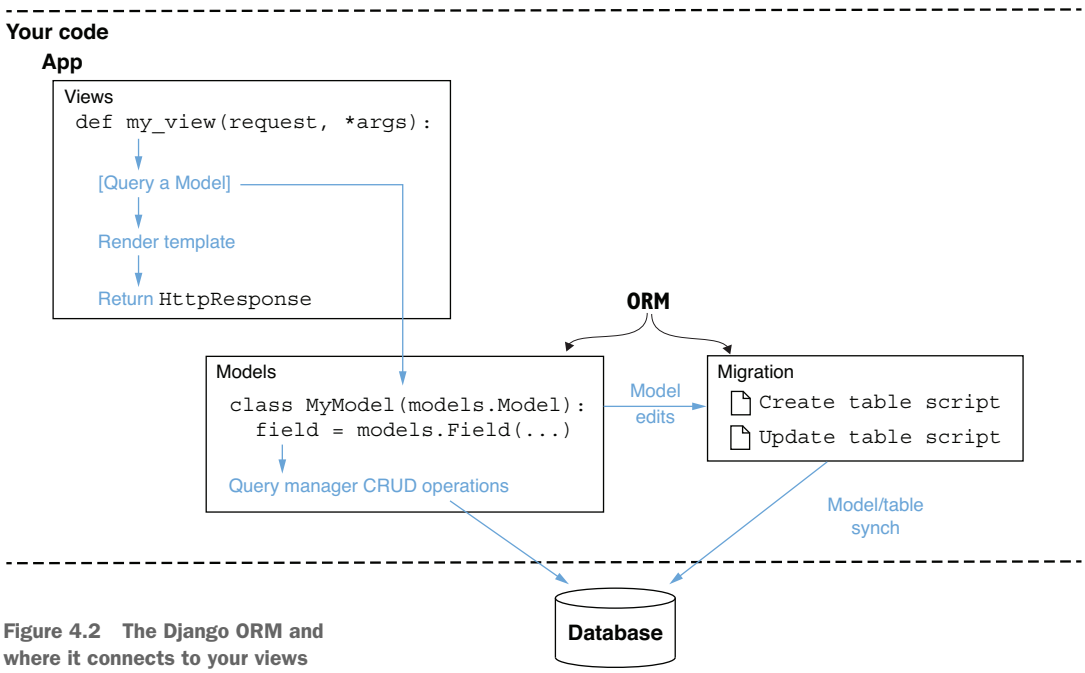



**Figure 4.2 The Django ORM and where it connects to your views**